

## Question 1 (Explanation) – OS HW-1 M. Zayed Nauman, 29047

```
42 int
43 main(void)
44 {
45     int i, j, k;
46
47     init_matrices();
48
49     //For each row, create a fresh pipe, fork child, child writes row to pipe
50     //parent immediately reads row and waits for child. This keeps only few fds open
51     for (i = 0; i < N; i++) {
52         int p[2];
53         if (pipe(p) < 0) {
54             printf("pipe failed for row %d\n", i);
55             exit(1);
56         }
57
58         int pid = fork();
59         if (pid < 0) {
60             printf("fork failed\n");
61             exit(1);
62         }
63
64         if (pid == 0) {
65             //Child will compute row i and write it to pipe
66             close(p[0]); //close read end in child
67             int row[N];
68             for (j = 0; j < N; j++) {
69                 int sum = 0;
70                 for (k = 0; k < N; k++) {
71                     sum += A[i][k] * B[k][j];
72                 }
73                 row[j] = sum;
74             }
75             write(p[1], (char*)row, sizeof(row));
76         }
77         else {
78             //Parent will read the row from pipe and then wait for the child
79             close(p[1]);
80             int got = read(p[0], (char*)C[i], sizeof(C[i]));
81             if (got != sizeof(C[i])) {
82                 printf("parent: read row %d returned %d (expected %d)\n", i, got, (int)sizeof(C[i]));
83             }
84             close(p[0]);
85             wait(0);
86         }
87     }
88 }
```

I have created written a basic c code for matrix multiplication where 10x10 matrices get initialized and filled. To implement the multiplication according to xv6, for each row of the result matrix C, the program creates a new pipe and a new child process using fork() to make each child compute exactly one row of the resultant matrix

```
75 //Write full row
76 if (write(p[1], (char*)row, sizeof(row)) != sizeof(row)) {
77     printf("child %d: write error\n", i);
78 }
79 close(p[1]);
80 exit(0);
81 } else {
82     //Parent will read the row from pipe and then wait for the child
83     close(p[1]);
84     int got = read(p[0], (char*)C[i], sizeof(C[i]));
85     if (got != sizeof(C[i])) {
86         printf("parent: read row %d returned %d (expected %d)\n", i, got, (int)sizeof(C[i]));
87     }
88     close(p[0]);
89     wait(0);
90 }
91 }
```

Printing and end of process

